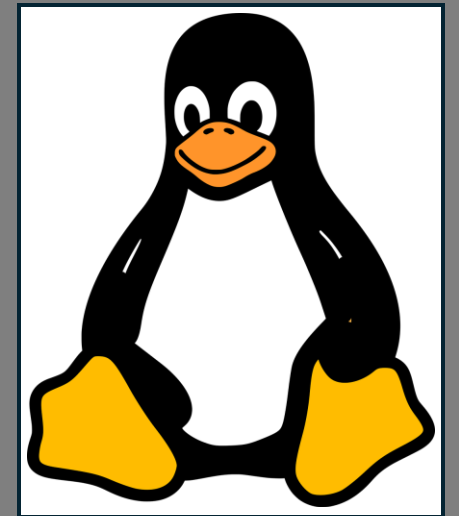


A Crash Course in Linux

(It's not that scary, promise)

Tux

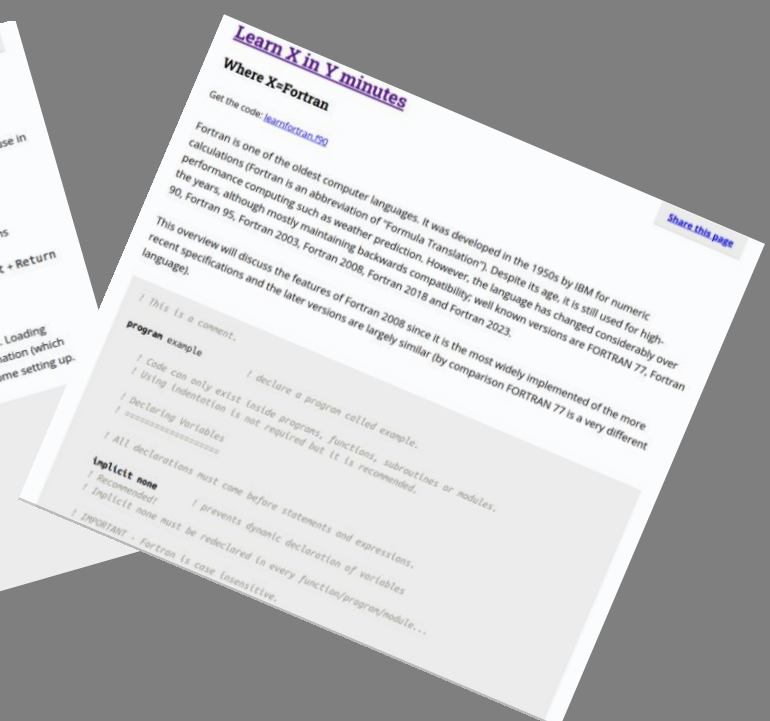
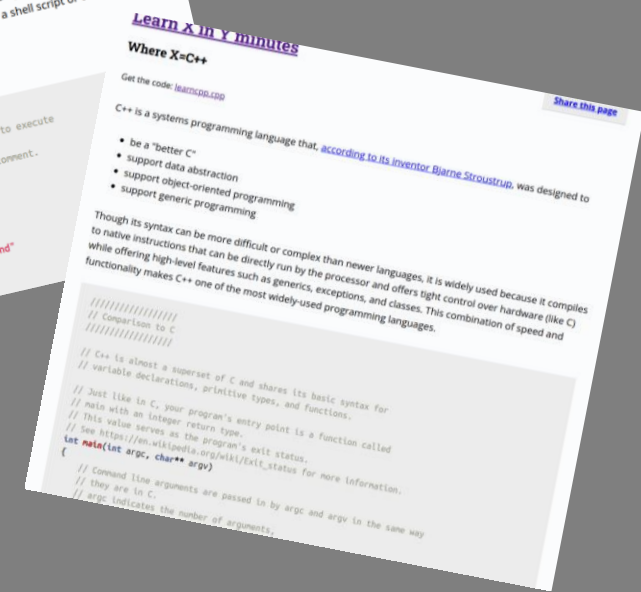
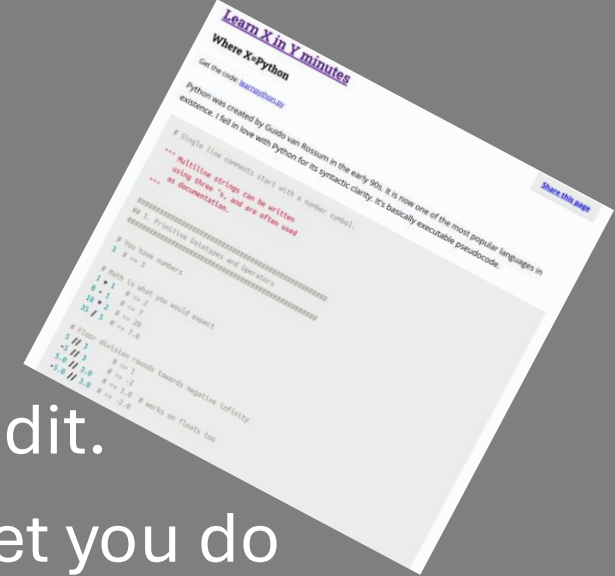


Resources – find the best way to learn for you

- [CCC GitHub repository](#)
- [Boot.dev free-to-read course](#)
 - Alternatively, their YouTube channel and the [course video](#), where you can see the correct answer to the exercises. Also, it's only 2 hrs 30 mins!
 - If you want to do the exercises, you'll need this repo: <https://github.com/bootdotdev/worldbanc>
- [Ubuntu tutorial](#)
- [Linux Journey](#)
 - Free (I think) lessons with exercises and answer checks.
- Command explanations:
 - [Explainshell.org](#)
 - [LinuxCommand.org](#)
- If you've got an ad blocker, try [Linux Survival](#)
- Written tutorials:
 - <https://ryanstutorials.net/linuxtutorial/>
 - <https://lym.readthedocs.io/en/latest/>
 - <https://learnxinyminutes.com/bash/>
- Cheat sheets are out there too.

<https://learnxinyminutes.com/>

- Use this if your supervisor gives you ancient code to edit.
- Hopefully, you can get enough knowledge quickly to let you do your work. SLIDE BETTER IN GENERAL CAPT SLIDES



You can actually code in a terminal!

- To print, type: `$ echo "Hello world"`
- To see your username: `$ whoami`
- A terminal, shell command line/prompt and CLI all refer to a program that allows you to interact with a computer in a text-based way.
- A CLI gives you more control over what you're doing.
- The commands you can use in the terminal are determined by the shell.

What's a shell?

- The shell runs the commands written in the terminal.
- They do the following:
 - Read the commands typed.
 - Evaluate those commands by running other programs.
 - Print the output of those commands.
 - Give you a new prompt to type another command.
 - Repeat.
- In Ubuntu, the shell is called Bash (macOS is zsh).

Programs can be written in shell languages...

- Please don't make big programs though – it's better for running other programs and making small scripts.
- You will probably encounter Bash scripts (.sh) if you need to use the HPC.
- You can create a local variable: `$ name="Lauren"` (no spaces)
- Then to print it: `$ echo $name` (`$name` lets you use the variable)
- Use the variable in a string: `$ echo Hello $name`

Command history

- To see your shell command history: \$ **history**
- Otherwise, with the prompt empty, use the "up" arrow key to cycle through your history.
 - The "down" arrow will go back down.
 - Very good for when you can't remember long commands you use regularly.
- To clear your terminal of text (*not delete* your history) use: \$ **clear**
 - Or ctrl + l

Terminal alternatives

- There are alternatives such as in VS Code (or Zed or Cursor).
- VS Code is good, and we will discuss setting it up.
- If you don't want Microsoft seeing your stuff, Zed looks very similar.

Filesystems

- We know data on a computer is stored in the filesystem.
- It's just a tree of directories (folders) and files.
- The root directory is the start of the tree and is often "home".
 - Your terminal should open here.
- Use the "print working directory" command to see your current working directory absolute filepath: \$ `pwd`

Filesystems

- To see the contents of your current working directory, use the "list" command: \$ `ls`
 - If you want more detail run \$ `ls -lrt`
 - `-l` shows the output as a "long" list
 - `-r` shows it in reverse order
 - `-t` shows it by last modification time
 - `-a` will show "all" files including hidden files and directories
- Most commands have these extra arguments for more options.

Directories

- To "change directory" (move into a folder): `$ cd folder1`
 - Don't forget you can tab multiple times to autocomplete (or be shown all matching options)
- To move out of the current directory: `$ cd ..`
 - `..` just refers to the parent directory
- If you want to go back up by two directories: `$ cd ../..`
 - You can do this as many times as you want (you can't go back past the root however).
- If you want to go back to your home directory use the tilde `~`:
 - `$ cd ~`
- To make a directory, use the "make directory" command:
 - `$ mkdir my_directory`

Files

- The `cat` command is used to view files.
 - Stands for "concatenate" as it can be used to view multiple files at once.
- Use as follows:
 - One file: `$ cat file1.txt`
 - Concatenate the contents of two files: `$ cat file1.txt file2.txt`
- To view the start of a file: `$ head -n 10 file1.txt`
 - Prints first `n` lines of the file (if no number is specified, defaults to `10`).
- To view the end of a file: `$ tail -n 10 file1.txt`

Files

- To view the contents of a file one page (or line) at a time, use **more** and **less**.
 - Instead of cat printing the whole file (which could be a lot)
- Actually, only use **less** as it does everything **more** does and has more features.
- Simply use as: `$ less data.csv`
 - The `-N` flag shows line numbers: `$ less -N data.csv`
 - Use "enter" to scroll down a few lines.
 - "spacebar" scrolls one page at a time.
 - "b" goes back a page.
 - "q" exits the program.

Files

- The `touch` command updates access to a file
- Helpfully, it will create an empty file with the given filename if it doesn't exist.
- Usage:
 - `$ touch new_file.txt`
 - `$ touch a_new_file.txt another_new_file.txt`

Moving stuff

- The "move" command moves a file or directory.
- You can also use it to rename a file.
 - `$ mv some_file.txt some_other_name.txt`
- Move file from current directory to another directory:
 - `$ mv some_file.txt some_directory/some_file.txt`
- Move file from current directory to parent directory:
 - `$ mv some_file.txt ../some_file.txt`
- If you don't want to rename, just move:
 - `$ mv some_file.txt some_directory/`

Removing stuff

- The "remove" command deletes a file or empty directory:
 - `$ rm some_file.txt`
- To delete a directory and all of its contents recursively, use the flag `-r`:
 - `$ rm -r some_directory`
- To force it (no confirmation), use the `-f` flag (be careful with this one):
 - `$ rm -rf some_directory`

Copying stuff

- The "copy" command copies a file from one location to another (naturally):
 - `$ cp source_file.txt destination/`
- Again, copy the directory and all its contents recursively using the flag `-R`:
 - `$ cp -R my_dir new_dir`

Search for text in files

- The `grep` command lets you search for text in files.
- Search for a string in a file to print out every line in the file that contains the string (case sensitive):
 - `$ grep "hello" words.txt`
- Search multiple files at once:
 - `$ grep "hello" hello.txt hello2.txt`
- Search an entire directory (recursively):
 - `$ grep -r "hello" .`
 - `.` is the current directory

Find files and directories by name

- Use the `find` command.
- Look for a file called `hello.txt`:
 - `$ find some_directory -name hello.txt`
- Search for files that match a pattern, e.g. all `.txt` files:
 - `$ find some_directory -name "*.txt"`
 - `*` matches anything
- Filenames containing a specific word:
 - `$ find some_directory -name "*lauren*"`

Superuser

- That should be you, which means you can use **sudo** ("superuser do") to run a command as one.
- Be careful when running commands with **sudo**.
- It asks for you password before completing the command.
- Sometimes you need it but check before you use it.

Executables

- Files with the `.sh` extension are shell scripts – text files that contain shell commands.
- Just run a shell script by typing its filepath:
 - `$ mydir/program.sh`
- If the script is in the current directory, you must prefix it with `./`:
 - `$ ./program.sh`
 - This just tells the shell we're running a file from a path, not an installed command like `ls`.
- You will need these files when using the HPC.

Executables

- To run a `.sh` file, you need to tell the computer what program to use to interpret a file.
- The line at the top of the script to tell your shell which program to use is called a "shebang".
- Format:
 - `#! interpreter [optional-arg]`
- For example, for a Python 3 script:
 - `#!/usr/bin/python3`

PATH

- Sometimes you will instal a program, but try to run it from the terminal and get an error:
 - `$ my-new-program`
 - `-bash: my-new-program: command not found`
- That's probably because it's not stored in the PATH variable, a list of directories the shell looks at when you try to run a command.
- You can edit this global variable using the `export` command:
 - `$ export PATH="$PATH:/path/to/new"`
 - This just adds your new path onto the end of the variable with `:` separating existing directories from the new directory.

PATH

- The previous commands only work for your current shell session.
- When you restart your shell, PATH will reset.
- To change it permanently, you need to add the command to your shell's configuration file (`./bashrc`).

Manual

- The `man` ("manual") command displays the manual for other programs (if available).
- The manual has a manual:
 - `$ man man`
- Navigation:
 - "spacebar" - next page. "b" - previous page.
 - "enter" - next line.
- To search for text:
 - "/" then start typing
 - "n" jumps to the next result. "N" goes back a result.
- "q" to quit.

Flags

- Some commands accept flags as you've seen (read their manuals).
- For example, with `ls`: `-l`, `-a`
 - You have also seen you can combine flags: `$ ls -al`
- Single-character flags are prefixed with a single dash (e.g. `-a`).
- Multi-character flags are prefixed with two dashes (e.g. `--help`).
- Sometimes the same flag can be used either way (e.g. `-h` or `--help`).

Help

- Most programs have a `help` option that prints a quick start guide instead of needing to look at the full manual.
- It should be accessed with one of:
 - Flag: `--help`
 - Flag: `-h`
 - First argument: `help`

Exit codes

- They just show you whether a program ran successfully or not.
- Somewhat confusingly (to me), `0` is the exit code for success, and `1` is an error.
- If it doesn't already provide you an exit code or obvious error, you can check if previously run program's exit code:
 - `$ echo $?`

Redirecting

- You can redirect the stdout (standard output, where programs print their output) and stderr (standard error, where programs print their error messages) into a different place.
 - This can be useful if you want to save some outputs to a text file, which aren't being saved in some Python code for example.
- To redirect stdout to a file:
 - `$ echo "Hello world" > hello.txt`
 - `$ cat hello.txt`
 - `# Hello world`
- To redirect stderr to a file:
 - `$ cat doesnotexist.txt 2> error.txt`
 - `$ cat error.txt`
 - `# cat: doesnotexist.txt: No such file or directory`

Piping

- In the shell, you can pipe the output of one program into the input of another with the operator `|`.
- For example:
 - `$ echo "Have you heard the tragedy of Darth Plagueis the Wise?" | wc -w`
 - This pipes the output into the word count command and prints the number of words received.

Top

- The **top** command lets you see what programs are using the most resources.
- This is useful locally, but especially important for using Captain/Ohdar to make sure you don't use too many resources.
- It's basically task manager but in the command line.

CLI text editor

- You might have heard of Vim...
 - *And also been stuck in it*
- To use it:
 - `$ vim FILEPATH_HERE`
- The most useful commands to me:
 - Use the arrow keys to move the cursor.
 - Enter insert mode to edit the file with 'i' (it should say --INSERT--) at the bottom.
 - Press 'esc' to return to normal mode.
 - Press ':wq' to save and exit (you can also just do ':w', then ':q')
- Cheat sheet: <https://vim.rtorr.com/>

Installing packages

- The package manager is apt.
- You will probably just need to copy commands.
- [Webi](#) can be used to install packages directly, with no need to use apt.
 - Be careful, it is the internet after all...
- For example, you could use it to install a cooler version of ls: [lsd](#).
 - *(It has icons and colours :))*